We initiate Keras with the PyTorch backend.

In [1]:
```
import os
os.environ["KERAS_BACKEND"] = "torch"

import keras

print("Keras version:", keras.__version__)
print("Backend:", keras.backend.backend())

keras.utils.set_random_seed(42)
```

```
Keras version: 3.13.2
Backend: torch
```

In [2]:
```
!pip install scikit-learn-intelex ucimlrepo torchinfo
```

```
Collecting scikit-learn-intelex
  Downloading scikit_learn_intelex-2026.2.0-py313-none-manylinux_2_28_x86_64.whl.
metadata (9.6 kB)
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: torchinfo in /usr/local/lib/python3.13/dist-packag
es (1.8.0)
Collecting daal==2026.2.0 (from scikit-learn-intelex)
  Downloading daal-2026.2.0-py2.py3-none-manylinux_2_28_x86_64.whl.metadata (1.1
kB)
Requirement already satisfied: numpy>=1.23.5 in /usr/local/lib/python3.13/dist-pa
ckages (from scikit-learn-intelex) (2.1.3)
Requirement already satisfied: scikit-learn>=1.0 in /usr/local/lib/python3.13/dis
t-packages (from scikit-learn-intelex) (1.6.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-packag
es (from scikit-learn-intelex) (26.3)
Collecting tbb==2023.* (from daal==2026.2.0->scikit-learn-intelex)
  Downloading tbb-2023.2.0-py2.py3-none-manylinux_2_28_x86_64.whl.metadata (1.0 k
B)
Requirement already satisfied: tcmlib==1.* in /usr/local/lib/python3.13/dist-pack
ages (from tbb==2023.*->daal==2026.2.0->scikit-learn-intelex) (1.5.0)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.13/dist-pa
ckages (from ucimlrepo) (2.3.3)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.13/di
st-packages (from ucimlrepo) (2026.7.22)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.1
3/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-pac
kages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-p
ackages (from pandas>=1.0.0->ucimlrepo) (2026.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-pac
kages (from scikit-learn>=1.0->scikit-learn-intelex) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-pa
ckages (from scikit-learn>=1.0->scikit-learn-intelex) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/
dist-packages (from scikit-learn>=1.0->scikit-learn-intelex) (3.6.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist
-packages (from joblib>=1.2.0->scikit-learn>=1.0->scikit-learn-intelex) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-package
s (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
Downloading scikit_learn_intelex-2026.2.0-py313-none-manylinux_2_28_x86_64.whl
(2.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.7/2.7 MB 29.1 MB/s eta 0:00:00a 0:0
0:01
Downloading daal-2026.2.0-py2.py3-none-manylinux_2_28_x86_64.whl (31.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 31.8/31.8 MB 50.6 MB/s eta 0:00:00:0
0:0100:01
Downloading tbb-2023.2.0-py2.py3-none-manylinux_2_28_x86_64.whl (6.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 72.2 MB/s eta 0:00:00:00:0
100:01
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: tbb, daal, ucimlrepo, scikit-learn-intelex
  Attempting uninstall: tbb
    Found existing installation: tbb 2022.3.1
    Uninstalling tbb-2022.3.1:
      Successfully uninstalled tbb-2022.3.1
ERROR: pip's dependency resolver does not currently take into account all the pac
kages that are installed. This behaviour is the source of the following dependenc
y conflicts.
```

```
mkl 2025.3.1 requires tbb==2022.*, but you have tbb 2023.2.0 which is incompatibl
e.
Successfully installed daal-2026.2.0 scikit-learn-intelex-2026.2.0 tbb-2023.2.0 u
cimlrepo-0.0.7
```

We enable the Intel Extensions for Scikit-Learn, which allows speedup for Sklearn operations:

In [3]:
```
import sys

# Kaggle-specific
sys.path.insert(
    0,
    "/usr/local/lib/python3.13/site-packages",
)

from sklearnex import patch_sklearn

patch_sklearn()
```

```
Extension for scikit-learn* enabled (https://github.com/uxlfoundation/scikit-lear
n-intelex)
```

Part A: Comparing Traditional Machine Learning Models

Step A.1: Load and Analyse Your Dataset

We load the dataset via the UCI ML repository; which already separates features and targets.

In [4]:
```
from ucimlrepo import fetch_ucirepo
import pandas as pd
import json

# fetch dataset
beijing_pm2_5 = fetch_ucirepo(id=381)

# data (as pandas dataframes)
X = pd.DataFrame(beijing_pm2_5.data.features )
y = pd.DataFrame(beijing_pm2_5.data.targets )

# metadata
print(json.dumps(beijing_pm2_5.metadata, indent = 2))

# variable information
print(beijing_pm2_5.variables)
```

```
{
  "uci_id": 381,
  "name": "Beijing PM2.5",
  "repository_url": "https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data",
  "data_url": "https://archive.ics.uci.edu/static/public/381/data.csv",
  "abstract": "This hourly data set contains the PM2.5 data of US Embassy in Beij
ing. Meanwhile, meteorological data from Beijing Capital International Airport ar
e also included. ",
  "area": "Climate and Environment",
  "tasks": [
    "Regression"
  ],
  "characteristics": [
    "Multivariate",
    "Time-Series"
  ],
  "num_instances": 43824,
  "num_features": 11,
  "feature_types": [
    "Integer",
    "Real"
  ],
  "demographics": [],
  "target_col": [
    "pm2.5"
  ],
  "index_col": [
    "No"
  ],
  "has_missing_values": "yes",
  "missing_values_symbol": "NaN",
  "year_of_dataset_creation": 2015,
  "last_updated": "Sat Mar 16 2024",
  "dataset_doi": "10.24432/C5JS49",
  "creators": [
    "Song Chen"
  ],
  "intro_paper": {
    "ID": 432,
    "type": "NATIVE",
    "title": "Assessing Beijing's PM2.5 pollution: severity, weather impact, APEC
and winter heating",
    "authors": "Xuan Liang, T. Zou, Bin Guo, Shuo Li, Haozhe Zhang, Shuyi Zhang,
Hui Huang, S. Chen",
    "venue": "Proceedings of the Royal Society A",
    "year": 2015,
    "journal": null,
    "DOI": "10.1098/rspa.2015.0257",
    "URL": "https://www.semanticscholar.org/paper/8a82cccc111cbe3e7ff7bc16a3345af
e8351a425",
    "sha": null,
    "corpus": null,
    "arxiv": null,
    "mag": null,
    "acl": null,
    "pmid": null,
    "pmcid": null
  },
  "additional_info": {
    "summary": "The data's time period is between Jan 1st, 2010 to Dec 31st, 201
```

```
4. Missing data are denoted as \"NA\".",
    "purpose": null,
    "funded_by": null,
    "instances_represent": null,
    "recommended_data_splits": null,
    "sensitive_data": null,
    "preprocessing_description": null,
    "variable_info": "No: row number\r\nyear: year of data in this row\r\nmonth:
month of data in this row\r\nday: day of data in this row\r\nhour: hour of data i
n this row\r\npm2.5: PM2.5 concentration (ug/m^3)\r\nDEWP: Dew Point (\u00e2\u201
e\u0192)\r\nTEMP: Temperature (\u00e2\u201e\u0192)\r\nPRES: Pressure (hPa)\r\ncbw
d: Combined wind direction\r\nIws: Cumulated wind speed (m/s)\r\nIs: Cumulated ho
urs of snow\r\nIr: Cumulated hours of rain\r\n",
    "citation": null
  }
}
```


	name	role	type	demographic	description	units	missing_values
0	No	ID	Integer	None	None	None	no
1	year	Feature	Integer	None	None	None	no
2	month	Feature	Integer	None	None	None	no
3	day	Feature	Integer	None	None	None	no
4	hour	Feature	Integer	None	None	None	no
5	pm2.5	Target	Integer	None	None	None	yes
6	DEWP	Feature	Integer	None	None	None	no
7	TEMP	Feature	Integer	None	None	None	no
8	PRES	Feature	Integer	None	None	None	no
9	cbwd	Feature	Categorical	None	None	None	no
10	Iws	Feature	Continuous	None	None	None	no
11	Is	Feature	Integer	None	None	None	no
12	Ir	Feature	Integer	None	None	None	no

Step A.2: Preprocess Your Data

First, we remove missing values.

In [5]:
```
X = X.copy()
y = y.copy()

valid = y["pm2.5"].notna()

# the pd.DataFrame is mostly to enforce type for my LSP
X = pd.DataFrame(X.loc[valid].copy())
y = pd.DataFrame(y.loc[valid].copy())

print("Original observations:", len(valid))
print("Valid observations:", valid.sum())
print("Missing targets:", (~valid).sum())
```

```
Original observations: 43824
Valid observations: 41757
Missing targets: 2067
```

We create the features engineered in the EDA. This include dewpoint spread, cyclical time encoding, one-hot encoding the `cbwd` feature, `log_Iws`, and relative humidity.

The dataset is already split between features and targets.

In [6]:
```
import numpy as np

X_engineered = X.copy()

# cyclical time encoding

X_engineered["month_sin"] = np.sin(2 * np.pi * X["month"] / 12)
X_engineered["month_cos"] = np.cos(2 * np.pi * X["month"] / 12)

X_engineered["hour_sin"] = np.sin(2 * np.pi * X["hour"] / 24)
X_engineered["hour_cos"] = np.cos(2 * np.pi * X["hour"] / 24)
X_engineered["log_Iws"] = np.log1p(X["Iws"])
X_engineered["log_Iws"].describe()


b = 17.625
c = 243.04

T = X["TEMP"]
D = X["DEWP"]

X_engineered["RH"] = 100 * np.exp(
    (b * D / (c + D))
    - (b * T / (c + T))
)

X_engineered = pd.get_dummies(
    X_engineered,
    columns=["cbwd"],
    dtype=int
)


X_engineered
```

Out[6]:

	year	month	day	hour	DEWP	TEMP	PRES	Iws	Is	Ir	month_sin
24	2010	1	2	0	-16	-4.0	1020.0	1.79	0	0	5.000000e-01
25	2010	1	2	1	-15	-4.0	1020.0	2.68	0	0	5.000000e-01
26	2010	1	2	2	-11	-5.0	1021.0	3.57	0	0	5.000000e-01
27	2010	1	2	3	-7	-5.0	1022.0	5.36	1	0	5.000000e-01
28	2010	1	2	4	-7	-5.0	1022.0	6.25	2	0	5.000000e-01
...	...	...	...	...	...	...	...	...	...	...	...
43819	2014	12	31	19	-23	-2.0	1034.0	231.97	0	0	-2.449294e-16
43820	2014	12	31	20	-22	-3.0	1034.0	237.78	0	0	-2.449294e-16
43821	2014	12	31	21	-22	-3.0	1034.0	242.70	0	0	-2.449294e-16
43822	2014	12	31	22	-22	-4.0	1034.0	246.72	0	0	-2.449294e-16
43823	2014	12	31	23	-21	-3.0	1034.0	249.85	0	0	-2.449294e-16

41757 rows × 20 columns

Step A.3: Train Two Traditional ML Models

We create a `results` dictionary to save results.

In [7]:
```
results = {}
```

We train the Random Forest Regressor *before* scaling; as scaling is unnecessary for that model.

So we create splits first to train Random Forest, then scale later for MLP and LSTM. For simplicity, we use the default hyperparameters for the Random Forest.

In [8]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_err



X_train, X_test, y_train, y_test = train_test_split(
    X_engineered, y,
```

```
    test_size=0.2,
    random_state=42,
    # shuffle = False
)

rf = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

# sklearnex requires this
y_train = y_train.to_numpy().ravel()
y_test = y_test.to_numpy().ravel()

rf.fit(X_train, y_train)

pred = rf.predict(X_test)

rf_results = {
    "RMSE": root_mean_squared_error(y_test, pred),
    "MAE": mean_absolute_error(y_test, pred),
    "R^2": r2_score(y_test, pred),
}

rf_results
```

Out[8]:
```
{'RMSE': 41.36466180346098, 'MAE': 24.81550287356322, 'R^2': 0.80581826801814}
```

In [9]:
```
from sklearn.model_selection import cross_validate


rf_cv_results = cross_validate(
    rf, X_train, y_train,
    cv = 5, n_jobs = -1,
    scoring={
        "R2": "r2",
        "RMSE": "neg_root_mean_squared_error",
        "MAE": "neg_mean_absolute_error"
})

rf_cv_summary = {
    "CV R^2": rf_cv_results["test_R2"].mean(),
    "CV RMSE": -rf_cv_results["test_RMSE"].mean(),
    "CV MAE": -rf_cv_results["test_MAE"].mean()
}

rf_results.update(rf_cv_summary)
results["RF"] = rf_results


rf_cv_summary
```

Out[9]:
```
{'CV R^2': np.float64(0.7964833258029169),
 'CV RMSE': np.float64(41.29443831344546),
 'CV MAE': np.float64(26.217336177218975)}
```

Now, we scale the dataset for MLP and LSTM.

In [10]:
```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Separate scalers for features and target
x_scaler = StandardScaler().set_output(transform="pandas")
y_scaler = StandardScaler()

# Chronological split
X_train, X_test, y_train, y_test = train_test_split(
    X_engineered,
    y,
    test_size=0.2,
)

# Scale features
X_train = x_scaler.fit_transform(X_train)
X_test = x_scaler.transform(X_test)

# Scale target
y_train_scaled = y_scaler.fit_transform(
    np.asarray(y_train).reshape(-1, 1)
).ravel()

y_test_scaled = y_scaler.transform(
    np.asarray(y_test).reshape(-1, 1)
).ravel()

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train_scaled.shape}")
print(f"y_test shape: {y_test_scaled.shape}")
```

```
X_train shape: (33405, 20)
X_test shape: (8352, 20)
y_train shape: (33405,)
y_test shape: (8352,)
```

Then we train the MLP model, using the default hyperparameters in the example notebook.

In [11]:
```
from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)
mlp.fit(X_train, y_train_scaled)

y_pred_scaled = mlp.predict(X_test)

y_pred = y_scaler.inverse_transform(
    y_pred_scaled.reshape(-1, 1)
).ravel()

mlp_results = {
    "RMSE": root_mean_squared_error(y_test, y_pred),
    "MAE": mean_absolute_error(y_test, y_pred),
    "R^2": r2_score(y_test, y_pred),
}
```

```
mlp_results
```

Out[11]:
```
{'RMSE': 48.24710784623302,
 'MAE': 32.55160628908441,
 'R^2': 0.7362317582954959}
```

MLP CV requires a more complex pipeline:

In [12]:
```
# Original unscaled data, for CV

from sklearn.compose import TransformedTargetRegressor
from sklearn.pipeline import make_pipeline

X_train_raw, X_test_raw, y_train_raw, y_test_raw = train_test_split(
    X_engineered,
    y,
    test_size=0.2,
    shuffle=True,
    random_state=42,
)


mlp_cv_model = TransformedTargetRegressor(
    regressor=make_pipeline(
        StandardScaler(),
        MLPRegressor(
            hidden_layer_sizes=(100, 50),
            max_iter=500,
            random_state=42,
        ),
    ),
    transformer=StandardScaler(),
)

mlp_cv_results = cross_validate(
    mlp_cv_model,
    X_train_raw,
    y_train_raw,
    cv=5,
    scoring={
        "R2": "r2",
        "RMSE": "neg_root_mean_squared_error",
        "MAE": "neg_mean_absolute_error",
    },
    n_jobs=-1,
)

mlp_cv_summary = {
    "CV R^2": mlp_cv_results["test_R2"].mean(),
    "CV RMSE": -mlp_cv_results["test_RMSE"].mean(),
    "CV MAE": -mlp_cv_results["test_MAE"].mean()
}

mlp_results.update(mlp_cv_summary)

results["MLP"] = mlp_results

mlp_cv_summary
```

Out[12]:
```
{'CV R^2': np.float64(0.720332276538023),
 'CV RMSE': np.float64(48.42500816176842),
 'CV MAE': np.float64(33.1265079835101)}
```

Step A.4: Evaluate and Compare Models

We obtain the results for the two models above

In [13]:
```
results
```

Out[13]:
```
{'RF': {'RMSE': 41.36466180346098,
  'MAE': 24.81550287356322,
  'R^2': 0.80581826801814,
  'CV R^2': np.float64(0.7964833258029169),
  'CV RMSE': np.float64(41.29443831344546),
  'CV MAE': np.float64(26.217336177218975)},
 'MLP': {'RMSE': 48.24710784623302,
  'MAE': 32.55160628908441,
  'R^2': 0.7362317582954959,
  'CV R^2': np.float64(0.720332276538023),
  'CV RMSE': np.float64(48.42500816176842),
  'CV MAE': np.float64(33.1265079835101)}}
```

We preview the results:

In [14]:
```
results_df = pd.DataFrame(results)
results_df.T
```

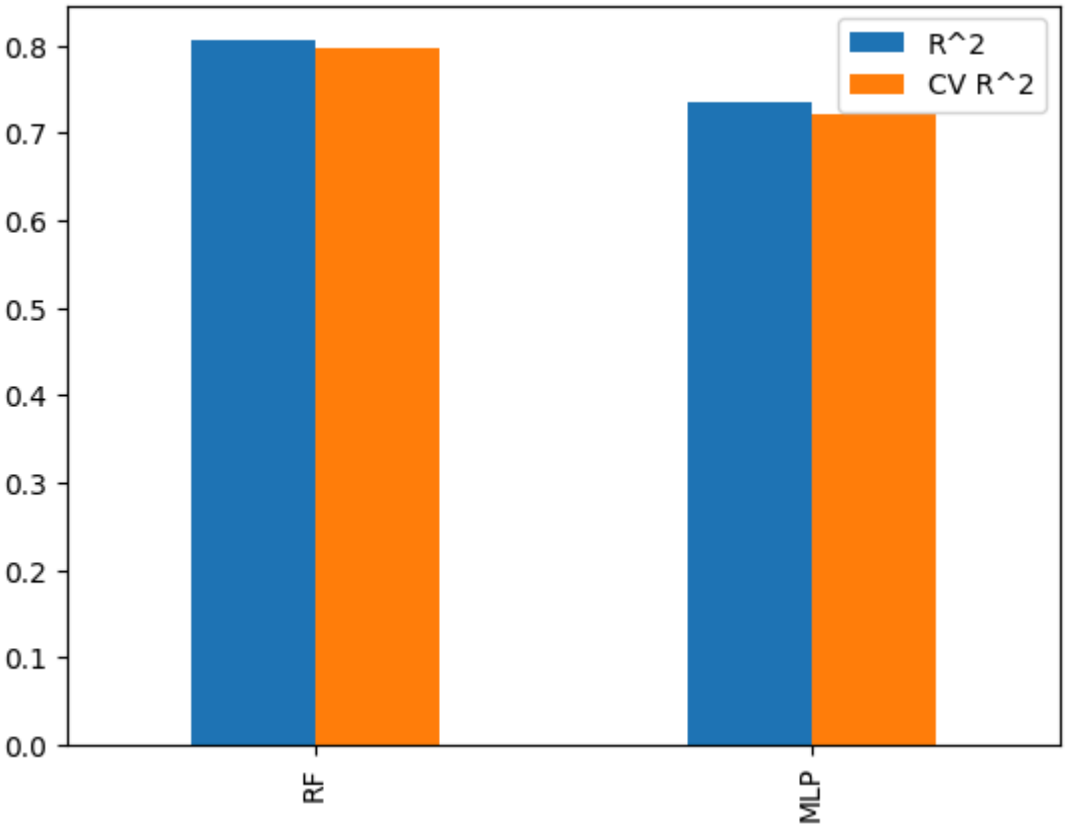
Out[14]:

	RMSE	MAE	R^2	CV R^2	CV RMSE	CV MAE
RF	41.364662	24.815503	0.805818	0.796483	41.294438	26.217336
MLP	48.247108	32.551606	0.736232	0.720332	48.425008	33.126508

Step A.5: Visualise Results

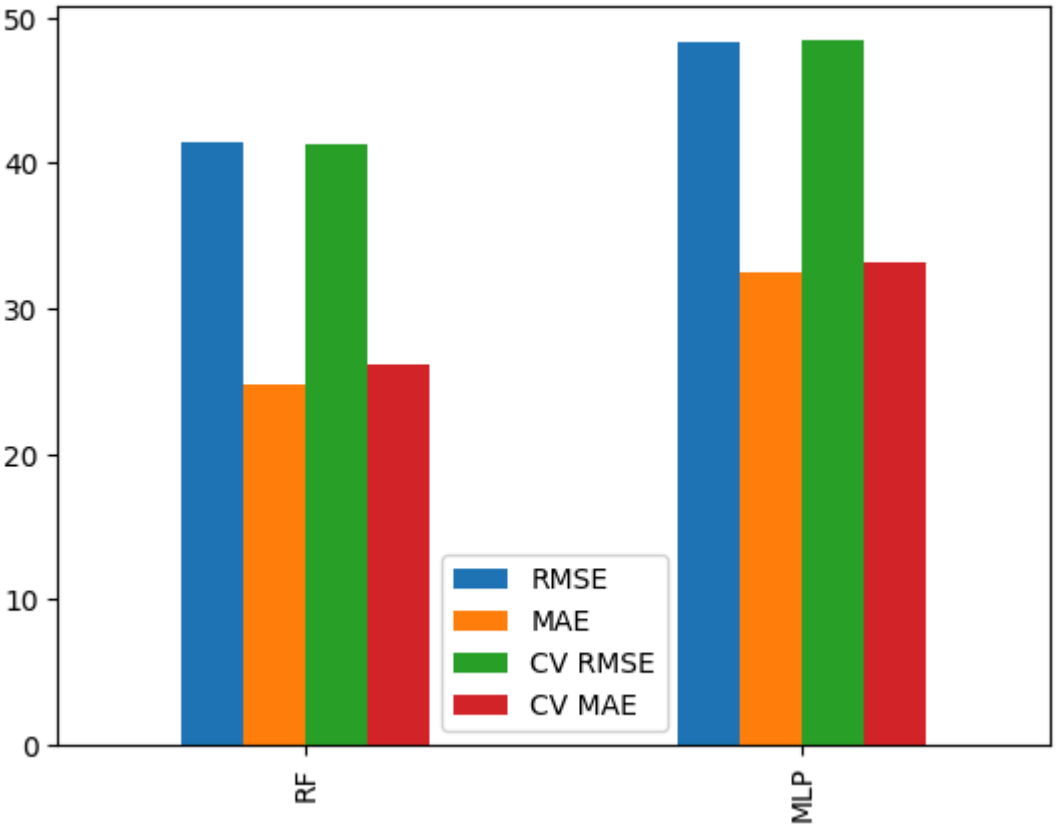
In [15]:
```
results_df.T[["R^2", "CV R^2"]].plot.bar()
```

Out[15]:
```
<Axes: >
```



In [16]:
```
results_df.T[["RMSE", "MAE", "CV RMSE", "CV MAE"]].plot.bar()
```

Out[16]: <Axes: >



Part B: Deep Learning - LSTM for Regression

Step B.1: Prepare Sequential Data

We split and scale the data.

In [17]:
```
from sklearn.preprocessing import MinMaxScaler

X_train, X_test, y_train, y_test = train_test_split(X_engineered, y,
    test_size=0.2,
    shuffle=False
)

lstm_x_scaler = MinMaxScaler().set_output(transform="pandas")
lstm_y_scaler = MinMaxScaler()

X_train = lstm_x_scaler.fit_transform(X_train)
X_test = lstm_x_scaler.transform(X_test)

y_train_scaled = lstm_y_scaler.fit_transform(
    np.asarray(y_train).reshape(-1, 1)
).ravel()

y_test_scaled = lstm_y_scaler.transform(
    np.asarray(y_test).reshape(-1, 1)
).ravel()

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train_scaled.shape}")
print(f"y_test shape: {y_test_scaled.shape}")
```

```
X_train shape: (33405, 20)
X_test shape: (8352, 20)
y_train shape: (33405,)
y_test shape: (8352,)
```

We use a sequence length of 20. Note that there are time gaps; so we skip windows with time gaps in them.

In [18]:
```
sequence_length = 20

def create_sequences(X, y, timestamps, sequence_length: int):
    X = np.asarray(X)
    y = np.asarray(y).ravel()
    timestamps = pd.DatetimeIndex(timestamps)

    X_sequences = []
    y_sequences = []

    for i in range(sequence_length, len(X)):
        # Include the target timestamp in the continuity check
        window_times = timestamps[i - sequence_length:i + 1]

        if not (window_times.to_series().diff().dropna()
                == pd.Timedelta(hours=1)).all():
            continue

        X_sequences.append(X[i - sequence_length:i])
```

```
        y_sequences.append(y[i])

    return np.asarray(X_sequences), np.asarray(y_sequences)

split_index = len(X_train)

timestamps = pd.to_datetime(
    X_engineered[["year", "month", "day", "hour"]]
)
train_timestamps = timestamps.iloc[:split_index]
test_timestamps = timestamps.iloc[split_index:]

X_train_windows, y_train_windows = create_sequences(
    X_train.to_numpy(),
    y_train_scaled,
    train_timestamps,
    sequence_length
)

X_test_windows, y_test_windows = create_sequences(
    X_test.to_numpy(),
    y_test_scaled,
    test_timestamps,
    sequence_length
)

print(X_train_windows.shape)
print(X_test_windows.shape)
print(y_train_windows.shape)
print(y_test_windows.shape)
```

```
(30428, 20, 20)
(7753, 20, 20)
(30428,)
(7753,)
```

In [19]:
```
for name, arr in {
    "X_train": X_train_windows,
    "X_test": X_test_windows,
    "y_train": y_train_windows,
    "y_test": y_test_windows,
}.items():
    print(name, arr.shape, np.isfinite(arr).all())
```

```
X_train (30428, 20, 20) True
X_test (7753, 20, 20) True
y_train (30428,) True
y_test (7753,) True
```

Step B.2: Build LSTM Architecture

We build the LSTM architecture per the assignment. Before building the model, we set device to GPU.

In [20]:
```
import torch

print("CUDA available:", torch.cuda.is_available())
```

```
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
CUDA available: True
GPU: Tesla T4
```

Now we define our model!

In [21]:
```
import keras
from keras import layers

model = keras.Sequential([
    layers.Input(shape=X_train_windows.shape[1:]),
    layers.LSTM(64, return_sequences=True),
    layers.Dropout(0.2),
    layers.LSTM(32, return_sequences=False),
    layers.Dropout(0.2),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="linear"),
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	P
lstm (LSTM)	(None, 20, 64)	
dropout (Dropout)	(None, 20, 64)	
lstm_1 (LSTM)	(None, 32)	
dropout_1 (Dropout)	(None, 32)	
dense (Dense)	(None, 16)	
dense_1 (Dense)	(None, 1)	

Total params: 34,721 (135.63 KB)

Trainable params: 34,721 (135.63 KB)

Non-trainable params: 0 (0.00 B)

Compile with AdamW optimizer and MSE loss, with MAE metrics

In [22]:
```
def make_optimizer():
    return keras.optimizers.AdamW(
        learning_rate=0.001,
        weight_decay=0.01,
        clipnorm=1.0
    )

model.compile(
    optimizer=make_optimizer(),
    loss="mse",
    metrics=["mae"]
)
```

Step B.3: Model Training

We first initiate some callbacks, per the problem requirements

In [23]:
```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        min_delta=0.0,
        restore_best_weights=True
    ),

    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=5,
        min_lr=1e-6,
        verbose=1
    )
]
```

In [24]:
```
history = model.fit(
    X_train_windows,
    y_train_windows,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    shuffle=False,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 44s 108ms/step - loss: 0.0077 - mae: 0.0629 - val_lo
ss: 0.0081 - val_mae: 0.0761 - learning_rate: 0.0010
Epoch 2/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0059 - mae: 0.0532 - val_lo
ss: 0.0110 - val_mae: 0.0846 - learning_rate: 0.0010
Epoch 3/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0056 - mae: 0.0518 - val_lo
ss: 0.0064 - val_mae: 0.0606 - learning_rate: 0.0010
Epoch 4/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0054 - mae: 0.0508 - val_lo
ss: 0.0040 - val_mae: 0.0465 - learning_rate: 0.0010
Epoch 5/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0051 - mae: 0.0495 - val_lo
ss: 0.0032 - val_mae: 0.0403 - learning_rate: 0.0010
Epoch 6/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0049 - mae: 0.0485 - val_lo
ss: 0.0030 - val_mae: 0.0386 - learning_rate: 0.0010
Epoch 7/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0048 - mae: 0.0476 - val_lo
ss: 0.0032 - val_mae: 0.0396 - learning_rate: 0.0010
Epoch 8/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 106ms/step - loss: 0.0046 - mae: 0.0467 - val_lo
ss: 0.0031 - val_mae: 0.0393 - learning_rate: 0.0010
Epoch 9/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0045 - mae: 0.0462 - val_lo
ss: 0.0031 - val_mae: 0.0400 - learning_rate: 0.0010
Epoch 10/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0044 - mae: 0.0459 - val_lo
ss: 0.0030 - val_mae: 0.0401 - learning_rate: 0.0010
Epoch 11/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - loss: 0.0041 - mae: 0.0437
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0044 - mae: 0.0454 - val_lo
ss: 0.0030 - val_mae: 0.0403 - learning_rate: 0.0010
Epoch 12/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0040 - mae: 0.0437 - val_lo
ss: 0.0029 - val_mae: 0.0382 - learning_rate: 5.0000e-04
Epoch 13/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0040 - mae: 0.0434 - val_lo
ss: 0.0029 - val_mae: 0.0374 - learning_rate: 5.0000e-04
Epoch 14/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0039 - mae: 0.0430 - val_lo
ss: 0.0029 - val_mae: 0.0374 - learning_rate: 5.0000e-04
Epoch 15/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 106ms/step - loss: 0.0039 - mae: 0.0427 - val_lo
ss: 0.0029 - val_mae: 0.0372 - learning_rate: 5.0000e-04
Epoch 16/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 105ms/step - loss: 0.0038 - mae: 0.0422 - val_lo
ss: 0.0029 - val_mae: 0.0371 - learning_rate: 5.0000e-04
Epoch 17/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step - loss: 0.0036 - mae: 0.0409
Epoch 17: ReduceLROnPlateau reducing learning rate to 0.0002502500011874362 8.
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0038 - mae: 0.0421 - val_lo
ss: 0.0029 - val_mae: 0.0374 - learning_rate: 5.0000e-04
Epoch 18/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0036 - mae: 0.0410 - val_lo
ss: 0.0032 - val_mae: 0.0374 - learning_rate: 2.5000e-04
Epoch 19/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0036 - mae: 0.0406 - val_lo
```

```
ss: 0.0031 - val_mae: 0.0370 - learning_rate: 2.5000e-04
Epoch 20/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0035 - mae: 0.0404 - val_lo
ss: 0.0031 - val_mae: 0.0369 - learning_rate: 2.5000e-04
Epoch 21/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0035 - mae: 0.0402 - val_lo
ss: 0.0031 - val_mae: 0.0370 - learning_rate: 2.5000e-04
Epoch 22/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step - loss: 0.0034 - mae: 0.0392
Epoch 22: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 108ms/step - loss: 0.0035 - mae: 0.0402 - val_lo
ss: 0.0031 - val_mae: 0.0369 - learning_rate: 2.5000e-04
Epoch 23/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0034 - mae: 0.0393 - val_lo
ss: 0.0033 - val_mae: 0.0378 - learning_rate: 1.2500e-04
Epoch 24/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0033 - mae: 0.0390 - val_lo
ss: 0.0034 - val_mae: 0.0379 - learning_rate: 1.2500e-04
Epoch 25/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0033 - mae: 0.0389 - val_lo
ss: 0.0034 - val_mae: 0.0380 - learning_rate: 1.2500e-04
Epoch 26/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0033 - mae: 0.0387 - val_lo
ss: 0.0034 - val_mae: 0.0380 - learning_rate: 1.2500e-04
```

We check the learning history:

In [25]:
```
import pandas as pd

history_df = pd.DataFrame(history.history)

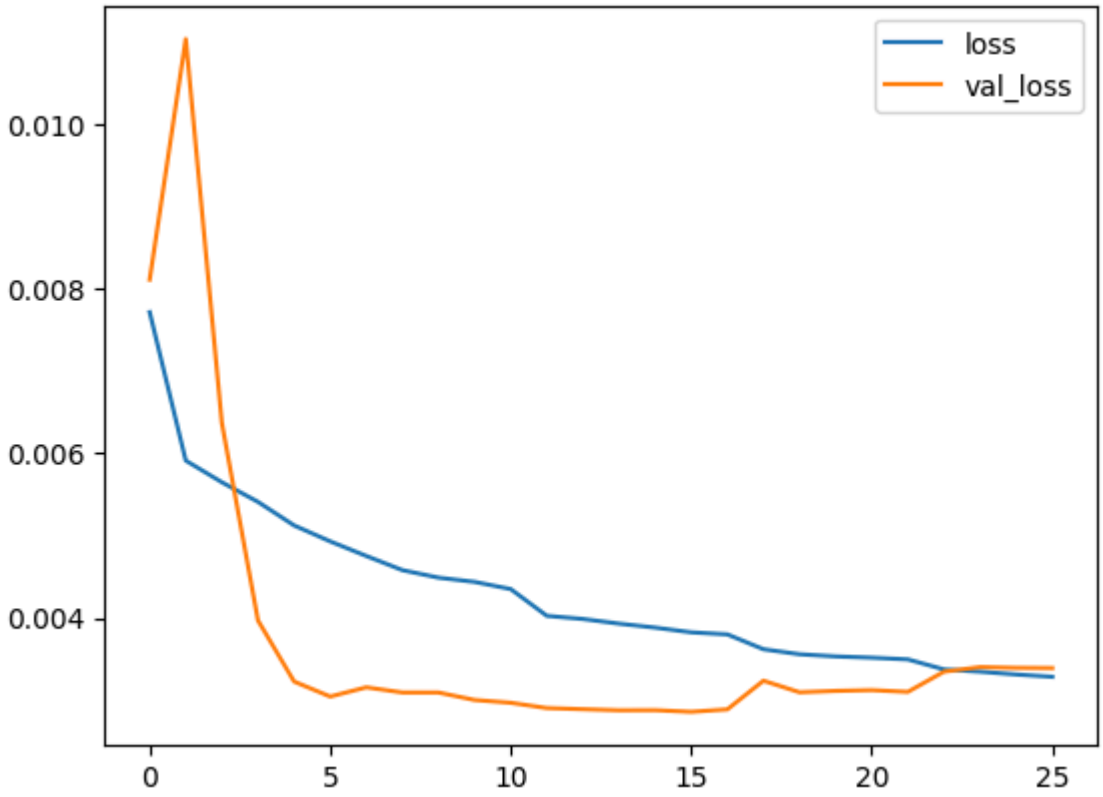
history_df.tail()
```

Out[25]:

	loss	mae	val_loss	val_mae	learning_rate
21	0.003495	0.040180	0.003100	0.036905	0.000250
22	0.003373	0.039297	0.003346	0.037772	0.000125
23	0.003346	0.039045	0.003402	0.037940	0.000125
24	0.003312	0.038896	0.003393	0.038028	0.000125
25	0.003284	0.038726	0.003390	0.038040	0.000125

In [26]:
```
history_df[["loss", "val_loss"]].plot()
```

Out[26]:
```
<Axes: >
```



Per the assignment requirements, we train another LSTM model

In [27]:
```
import keras
from keras import layers

model2 = keras.Sequential([
    layers.Input(shape=X_train_windows.shape[1:]),
    layers.LSTM(64, return_sequences=True),
    layers.Dropout(0.2),
    layers.LSTM(64, return_sequences=False),
    layers.Dropout(0.1),
    layers.Dense(64, activation="relu"),
    layers.Dense(1, activation="linear"),
])

model2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	P
lstm_2 (LSTM)	(None, 20, 64)	
dropout_2 (Dropout)	(None, 20, 64)	
lstm_3 (LSTM)	(None, 64)	
dropout_3 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 64)	
dense_3 (Dense)	(None, 1)	

Total params: 59,009 (230.50 KB)

Trainable params: 59,009 (230.50 KB)

Non-trainable params: 0 (0.00 B)

In [28]:
```
model2.compile(
    optimizer=make_optimizer(),
    loss="mse",
    metrics=["mae"]
)

history2 = model2.fit(
    X_train_windows,
    y_train_windows,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    shuffle=False,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0072 - mae: 0.0604 - val_lo
ss: 0.0062 - val_mae: 0.0642 - learning_rate: 0.0010
Epoch 2/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0058 - mae: 0.0527 - val_lo
ss: 0.0104 - val_mae: 0.0816 - learning_rate: 0.0010
Epoch 3/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0055 - mae: 0.0514 - val_lo
ss: 0.0117 - val_mae: 0.0848 - learning_rate: 0.0010
Epoch 4/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0053 - mae: 0.0501 - val_lo
ss: 0.0073 - val_mae: 0.0643 - learning_rate: 0.0010
Epoch 5/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step - loss: 0.0049 - mae: 0.0480
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0051 - mae: 0.0488 - val_lo
ss: 0.0042 - val_mae: 0.0473 - learning_rate: 0.0010
Epoch 6/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0045 - mae: 0.0465 - val_lo
ss: 0.0031 - val_mae: 0.0380 - learning_rate: 5.0000e-04
Epoch 7/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 106ms/step - loss: 0.0044 - mae: 0.0459 - val_lo
ss: 0.0028 - val_mae: 0.0363 - learning_rate: 5.0000e-04
Epoch 8/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 105ms/step - loss: 0.0043 - mae: 0.0450 - val_lo
ss: 0.0028 - val_mae: 0.0364 - learning_rate: 5.0000e-04
Epoch 9/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0042 - mae: 0.0445 - val_lo
ss: 0.0029 - val_mae: 0.0367 - learning_rate: 5.0000e-04
Epoch 10/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - loss: 0.0039 - mae: 0.0429
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0041 - mae: 0.0440 - val_lo
ss: 0.0029 - val_mae: 0.0362 - learning_rate: 5.0000e-04
Epoch 11/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0039 - mae: 0.0428 - val_lo
ss: 0.0029 - val_mae: 0.0355 - learning_rate: 2.5000e-04
Epoch 12/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0038 - mae: 0.0424 - val_lo
ss: 0.0029 - val_mae: 0.0352 - learning_rate: 2.5000e-04
Epoch 13/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0038 - mae: 0.0420 - val_lo
ss: 0.0030 - val_mae: 0.0356 - learning_rate: 2.5000e-04
Epoch 14/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0037 - mae: 0.0418 - val_lo
ss: 0.0030 - val_mae: 0.0356 - learning_rate: 2.5000e-04
Epoch 15/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step - loss: 0.0035 - mae: 0.0401
Epoch 15: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0037 - mae: 0.0414 - val_lo
ss: 0.0031 - val_mae: 0.0360 - learning_rate: 2.5000e-04
Epoch 16/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 40s 106ms/step - loss: 0.0035 - mae: 0.0405 - val_lo
ss: 0.0032 - val_mae: 0.0364 - learning_rate: 1.2500e-04
Epoch 17/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0035 - mae: 0.0402 - val_lo
ss: 0.0032 - val_mae: 0.0362 - learning_rate: 1.2500e-04
Epoch 18/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 41s 107ms/step - loss: 0.0035 - mae: 0.0401 - val_lo
ss: 0.0032 - val_mae: 0.0364 - learning_rate: 1.2500e-04
```

In [29]:
```
import pandas as pd

history2_df = pd.DataFrame(history2.history)

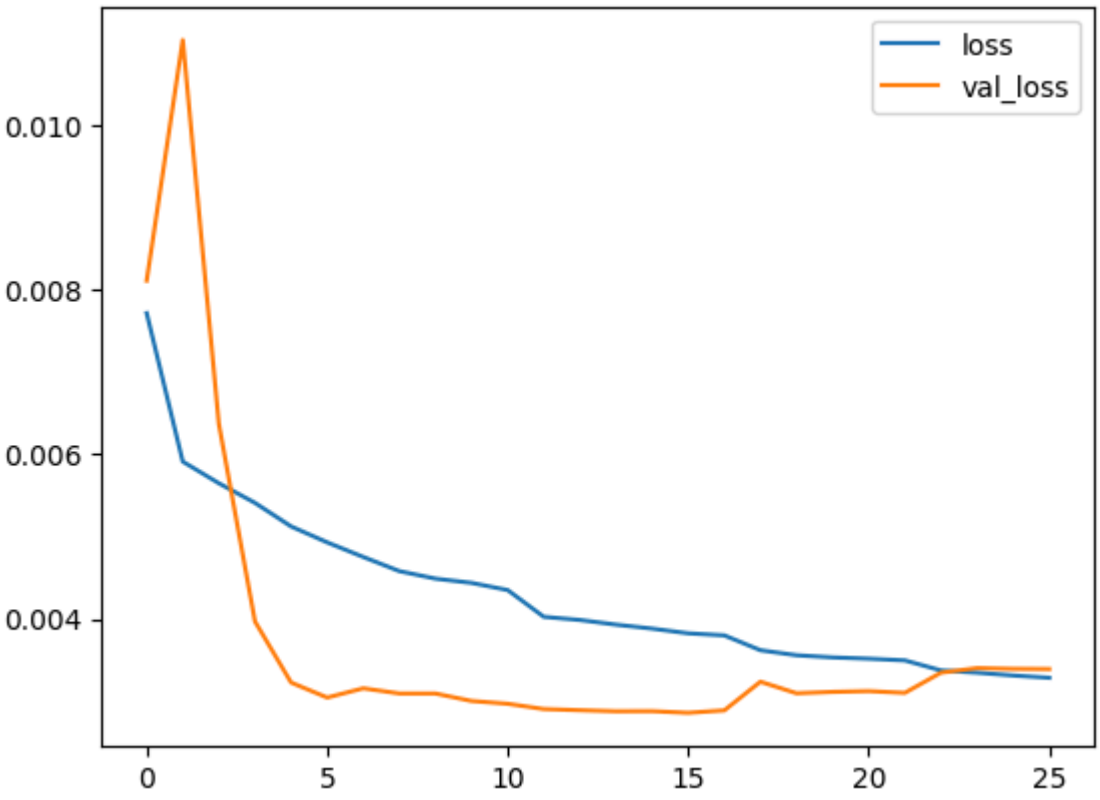
history2_df.tail()
```

Out[29]:

	loss	mae	val_loss	val_mae	learning_rate
13	0.003720	0.041829	0.003010	0.035601	0.000250
14	0.003671	0.041436	0.003091	0.035963	0.000250
15	0.003536	0.040541	0.003207	0.036434	0.000125
16	0.003494	0.040221	0.003170	0.036236	0.000125
17	0.003460	0.040098	0.003237	0.036386	0.000125

In [30]:
```
history_df2[["loss", "val_loss"]].plot()
```

Out[30]: <Axes: >



Step B.4: Evaluate the LSTM Model

We evaluate the two LSTM models.

In [31]:
```
y_pred_scaled = model.predict(X_test_windows)

# Convert predictions back to original PM2.5 units
y_pred = lstm_y_scaler.inverse_transform(
    y_pred_scaled.reshape(-1, 1)
).ravel()
```

```
# Convert the corresponding test targets back to original units
y_true = lstm_y_scaler.inverse_transform(
    y_test_windows.reshape(-1, 1)
).ravel()

assert len(y_true) == len(y_pred)

lstm_results = {
    "RMSE": root_mean_squared_error(y_true, y_pred),
    "MAE": mean_absolute_error(y_true, y_pred),
    "R^2": r2_score(y_true, y_pred),
}

results["LSTM1"] = lstm_results

lstm_results
```

```
243/243 ━━━━━━━━━━━━━━━━━━━━ 15s 60ms/step
```

Out[31]:
```
{'RMSE': 63.5176735451333, 'MAE': 44.43331227547951, 'R^2': 0.5417063488510893}
```

In [32]:
```
y_pred_scaled = model2.predict(X_test_windows)

# Convert predictions back to original PM2.5 units
y_pred = lstm_y_scaler.inverse_transform(
    y_pred_scaled.reshape(-1, 1)
).ravel()

# Convert the corresponding test targets back to original units
y_true = lstm_y_scaler.inverse_transform(
    y_test_windows.reshape(-1, 1)
).ravel()

assert len(y_true) == len(y_pred)

lstm2_results = {
    "RMSE": root_mean_squared_error(y_true, y_pred),
    "MAE": mean_absolute_error(y_true, y_pred),
    "R^2": r2_score(y_true, y_pred),
}

results["LSTM2"] = lstm2_results

lstm2_results
```

```
243/243 ━━━━━━━━━━━━━━━━━━━━ 15s 60ms/step
```

Out[32]:
```
{'RMSE': 68.18523393702495,
 'MAE': 47.525106567677554,
 'R^2': 0.4718766798238937}
```

Step B.5: Save the model

In [34]:
```
import pickle

# Save trained models
model.save("lstm1.h5")
model2.save("lstm2.h5")
```

```
# Save training histories
with open("lstm1_history.pkl", "wb") as f:
    pickle.dump(history.history, f)

with open("lstm2_history.pkl", "wb") as f:
    pickle.dump(history2.history, f)
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
```

Part C: Final Summary and Reflection

As the RF and MLP used shuffled datasets, compared to the LSTM used without, we re-evaluate them based on the non-shuffled datasets.

In [35]:
```
shuffled_results = {}

for key in ["RF", "MLP"]:
    shuffled_results[f"{key} (shuffled)"] = results.pop(key, None)
shuffled_results
```

Out[35]:
```
{'RF (shuffled)': {'RMSE': 41.36466180346098,
  'MAE': 24.81550287356322,
  'R^2': 0.80581826801814,
  'CV R^2': np.float64(0.7964833258029169),
  'CV RMSE': np.float64(41.29443831344546),
  'CV MAE': np.float64(26.217336177218975)},
 'MLP (shuffled)': {'RMSE': 48.24710784623302,
  'MAE': 32.55160628908441,
  'R^2': 0.7362317582954959,
  'CV R^2': np.float64(0.720332276538023),
  'CV RMSE': np.float64(48.42500816176842),
  'CV MAE': np.float64(33.1265079835101)}}
```

In [36]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X_engineered, y,
    test_size=0.2,
    # random_state=42,
    shuffle = False
)

rf = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

# sklearnex requires this
y_train = y_train.to_numpy().ravel()
y_test = y_test.to_numpy().ravel()

rf.fit(X_train, y_train)
```

```
pred = rf.predict(X_test)

rf_results = {
    "RMSE": root_mean_squared_error(y_test, pred),
    "MAE": mean_absolute_error(y_test, pred),
    "R^2": r2_score(y_test, pred),
}

results["RF"] = rf_results
rf_results
```

Out[36]:
```
{'RMSE': 67.83487776966848,
 'MAE': 47.39144516283525,
 'R^2': 0.4823174291685498}
```

In [37]:
```
x_scaler = StandardScaler().set_output(transform="pandas")
y_scaler = StandardScaler()

# Chronological split
X_train, X_test, y_train, y_test = train_test_split(
    X_engineered,
    y,
    test_size=0.2,
    shuffle = False
)

# Scale features
X_train = x_scaler.fit_transform(X_train)
X_test = x_scaler.transform(X_test)

# Scale target
y_train_scaled = y_scaler.fit_transform(
    np.asarray(y_train).reshape(-1, 1)
).ravel()

y_test_scaled = y_scaler.transform(
    np.asarray(y_test).reshape(-1, 1)
).ravel()

mlp = MLPRegressor(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)
mlp.fit(X_train, y_train_scaled)

y_pred_scaled = mlp.predict(X_test)

y_pred = y_scaler.inverse_transform(
    y_pred_scaled.reshape(-1, 1)
).ravel()

mlp_results = {
    "RMSE": root_mean_squared_error(y_test, y_pred),
    "MAE": mean_absolute_error(y_test, y_pred),
    "R^2": r2_score(y_test, y_pred),
}

results["MLP"] = mlp_results
mlp_results
```

Out[37]:
```
{'RMSE': 122.78100232315933,
 'MAE': 82.59353100149647,
 'R^2': -0.695976377264389}
```

Step C.1: Create Master Results Table

In [38]:
```
results
```

Out[38]:
```
{'LSTM1': {'RMSE': 63.5176735451333,
  'MAE': 44.43331227547951,
  'R^2': 0.5417063488510893},
 'LSTM2': {'RMSE': 68.18523393702495,
  'MAE': 47.525106567677554,
  'R^2': 0.4718766798238937},
 'RF': {'RMSE': 67.83487776966848,
  'MAE': 47.39144516283525,
  'R^2': 0.4823174291685498},
 'MLP': {'RMSE': 122.78100232315933,
  'MAE': 82.59353100149647,
  'R^2': -0.695976377264389}}
```

In [39]:
```
results_df = pd.DataFrame(results).T

results_df
```

Out[39]:

	RMSE	MAE	R^2
LSTM1	63.517674	44.433312	0.541706
LSTM2	68.185234	47.525107	0.471877
RF	67.834878	47.391445	0.482317
MLP	122.781002	82.593531	-0.695976

In [40]:
```
print(results_df.to_markdown())
```

```
|       |     RMSE |     MAE |       R^2 |
|:------|---------:|--------:|----------:|
| LSTM1 |  63.5177 | 44.4333 |  0.541706 |
| LSTM2 |  68.1852 | 47.5251 |  0.471877 |
| RF    |  67.8349 | 47.3914 |  0.482317 |
| MLP   | 122.781  | 82.5935 | -0.695976 |
```

We construct the following table:

Model Type	Model Name	R^2	RMSE
Traditional ML	Random Forest	0.48	67.83
Traditional ML	Multilayer Perceptron	-0.69	122.78
Deep Learning 1	LSTM	0.58	61.06
Deep Learning 2	LSTM	0.18	85.06

Step C.3: Answer Reflection Questions

1. For your deep learning model (CNN or LSTM): How did the architecture choices affect performance?

Adding more parameters in layers, and reducing dropout appears to reduce R^2 and increase RMSE loss. This suggest that increasing model complexity may reduce performance by issues such as overfitting.

2. Did your deep learning model outperform traditional ML models? Why or why not?

Yes. The best-performing LSTM achieved an R^2 of 0.5765 and an RMSE of 61.06, outperforming the Random Forest (R^2 = 0.4823, RMSE = 67.83) and MLP (R^2 = −0.6960, RMSE = 122.78). 8). One possible explanation is that LSTMs can learn temporal dependencies across sequential observations

3. If you had more time, what would you do to improve model performance further?

Given more time, I would perform systematic hyperparameter tuning, including experimenting with learning rates, batch sizes, dropout rates, and different sequence lengths.

Part D: GitHub Integration

This part is unavailable due to the retirement of GitHub Classroom.

Part E: GenAI Reflection (Mandatory)

GenAI Reflection

1. GenAI Tools Used

- ChatGPT: for debugging environment issues regarding Kaggle's cuML setup, and general code debugging**

2. Specific Tasks & Prompts

3. Code Attribution only for debugging and helping find errors

LSTM evaluation section: AI-assisted: Helped diagnose issues regarding target and prediction array shape mismatches

LSTM model training section: AI-assisted: Provided debugging guidance concerning optimizer configuration, model reinitialization, callbacks, and batch size.

4. Learning Reflection

- How did GenAI help me learn?

GenAI helped me understand the models, as well as evaluting errors and results.

- What challenges did it create?

AI-generated suggestions created challenges regarding hyperparameter tuning; as these require acutal training and supervision to evaluate

- What specific skills did I develop?

I developed skills to work with sklearn, Keras, and evaluating models.

In [41]:
```
# Part E: GenAI Reflection (Mandatory)
```